Choosing an inference library for Lege

tract, RTen and ONNX Runtime in a reproducible OCR pipeline

21CSM, with AI assistance (see the statement below)
Version 2.0 · 12 November 2026 · supersedes versions 1.0 and 1.1

Abstract. Lege, a command-line OCR tool, runs three small neural networks on every page. We compared three libraries for running them, tract, RTen and ONNX Runtime, inside Lege itself, on 25 reference images, on an Apple M3 Pro and on GitHub's Linux x86-64 and ARM64 runners, against a decision rule committed in advance. With PP-OCRv5, all three read the same text, peaked under 2 GB on the reference pages, and ran on emulated x86 CPUs without AVX. ONNX Runtime, in Microsoft's official build, was fastest on one Linux x86-64 thread by a factor of 1.80 over RTen (95% interval 1.76–1.84). The deciding requirement was the same output at every thread count. tract and RTen met it, and their raw model outputs were bit-identical at 1, 2, 4 and 8 threads. ONNX Runtime's raw outputs changed with the thread count on ARM64 and on the Mac, because one matrix-multiply kernel splits its sums differently across threads; the ALTO output hid this in most cases but not all. Running each model call on one thread and reading lines in parallel instead makes the output identical at any thread count by construction, for all three libraries; it takes about a third longer at 4 threads on the Mac and costs nothing on one thread. We choose ONNX Runtime, Microsoft's official build, with line-parallel threading and a check of raw outputs in continuous integration, on condition that its memory on full-size newspaper pages, which reached 2.07 GiB, is brought under the limit. A repeat with PP-OCRv6, labelled exploratory, found the full v6 models no better than v5 on Latin-script pages, a v6 detector with the v5 recognizer at least as good (too narrowly to act on), and tract over the memory limit with the medium models.

> **Authorship and AI assistance.** This study was designed and directed by 21CSM, who set the requirements and approved the decision. Claude Code (Anthropic, model Claude Opus 5.5) did most of the hands-on work: it wrote the comparison code and the benchmark and analysis scripts, ran the experiments, drafted this report and the accompanying blog post, and carried out the three reviews (an OCR expert, a Rust and performance expert, and a reviewer of scientific reports) whose 24 findings version 2.0 answers (Section 12). The reviews were adversarial self-reviews by the same assistant, not independent peer review. 21CSM read the results and the text; the tables are generated from the committed result files, so a number in the text can be traced to a measurement.

Contents

1 Introduction 2
2 Requirements and decision rule 2
3 Method 3
4 A defect found by the benchmark 4
5 Findings with PP-OCRv5 5
6 GPU and Neural Engine 11
7 Repeated with PP-OCRv6 (exploratory) 11
8 Threats to validity 15
9 Decision 16
10 Data availability and reproduction 17
11 Glossary 17
12 Changes between versions 18
References 18

1 Introduction

Lege turns a scanned page into ALTO 4.4 XML [1] and plain text. It is a command-line tool written in Rust, built for archives and for anyone who needs the same output every time. For each page it runs three PP-OCRv5 neural networks [2] in turn:

1. a detector finds the text lines on the page;
2. a direction classifier checks whether each line is upside down;
3. a recognizer reads the characters in each line.

Everything else, from decoding the image to writing ALTO, is Lege's own code. The one piece it borrows is the inference library that runs the networks. That library sets most of the speed and memory use, decides whether the output stays the same at every thread count, and decides what has to ship with Lege.

Lege started with tract [3], a pure-Rust ONNX runtime, without measuring the alternatives. This report measures tract against two others before that choice is made final:

- **RTen** [4], a younger pure-Rust runtime that loads ONNX files directly;
- **ONNX Runtime** [5], Microsoft's C++ runtime and the most widely used option, reached from Rust through the `ort` crate [6].

Terms that may be unfamiliar are defined in the glossary at the end.

2 Requirements and decision rule

Lege makes four promises that an inference library must keep. They became hard requirements; failing any one rules a library out:

1. **Same output at any thread count.** On one machine, the ALTO file is byte-identical at 1, 2, 3, 4, 6 and 8 threads, for every reference image.
2. **Runs on old CPUs.** Every page runs, and the text is right, on an x86 CPU without AVX and on one with AVX but no AVX2.
3. **Fits a 4 GB machine.** Peak memory stays under 2 GB (1.86 GiB) on every page.
4. **Same text.** The text matches the approved snapshot on every page.

Among libraries that pass, the ranking is: speed on one thread on Linux x86-64 (the closest stand-in for low-end machines, which have few cores), then speed at 4 threads on the Mac; totals within 10% count as a tie. Ties are broken by nothing extra to ship, then lower memory, then fewer added dependencies.

The rule was committed before the second round of Linux and emulation results came in, so that those data could not be read to fit a favourite.[1]

What was already known. The Mac results were known when the rule was written, and they included the failure of ONNX Runtime, as built by Nix, to keep the same output at 4 threads on the M3 Pro. The rule contains a clause that judges ONNX Runtime "by the build Lege would ship". That clause was written knowing it would keep ONNX Runtime in the comparison. Without it, ONNX Runtime fails the first requirement, and the rule picks RTen, which was 1.27× faster than tract on one Linux x86-64 thread in this version's measurements. The clause can be defended on engineering grounds, since Lege would never ship the Nix build, but the reader should know it was not written blind. Section 9 returns to this.

What was not committed in advance. Everything in Section 7 (PP-OCRv6), and the analyses added in version 2.0 (raw-output checks, line-parallel threading, build tuning, heap memory, newspapers, pooled error rates) were chosen after seeing version 1's results. They are exploratory.

[1] Commit `63ffb13` in the Lege repository, `docs/spikes/runtimes-decision-rule.md`, 2026-11-08 16:28 EST. A commit date can be set by its author, so this is a statement of practice, not proof.

3 Method

One code base, three builds. Lege keeps its inference calls behind four functions in one module. For this study a second implementation of those four functions was written for RTen and for ONNX Runtime, chosen by a build feature. Image preparation, box finding, character decoding and ALTO writing are the same code in all three builds, so the measurements reflect the real pipeline.

Library	Version	How Lege reaches it
tract	0.23.9	Rust crate, compiled in
RTen	0.27.0	Rust crate, compiled in
ONNX Runtime	1.27.1	`ort` 2.0.0-rc.15; C++ library loaded at run time

Table 1: The three libraries. All builds use Rust 1.99.1 in release mode and the same PP-OCRv5 mobile models, pinned by SHA-256.

Workload. The 25 images of Lege's acceptance tests: 20 scanned book pages printed 1871–1920 in English, French, German, Spanish and Polish (1200 pixels wide, 1 to 220 lines each) and 5 synthetic lines. Some images are the same page saved as another TIFF type or turned upside down, so there are about 17 distinct pages. Version 2.0 adds three full-size newspaper pages as a stress test (Section 5.9).

What the timers cover. Lege's `--timing` flag prints four numbers per page:

- `load`: reading the three model files and checking their SHA-256 hashes;
- `decode`: decoding the image;
- `recognize`: running the three networks and Lege's own pre- and post-processing. Each library also does its one-off setup here, the first time a model is used: parsing the ONNX file, optimizing the graph, and creating sessions or plans;
- `write`: rendering and writing the ALTO and text files.

The **page time** is decode + recognize + write, so it includes library setup but not `load`. Version 2.0 also records the **process time**: the wall-clock time of the whole `lege` process, from start to exit, as a user sees it. To separate setup from steady-state work, a variant reads a different page first, untimed, in the same process (Section 5.2).

Timing. One `lege` process per page, as people run it. Each page has a warm-up run and then three timed runs. The libraries take turns image by image and run by run, so a slowdown partway through affects all of them alike. Version 1 reported the sum of per-page medians, with the sums of per-page minima and maxima in brackets. That bracket is a range that was never itself observed. Version 2.0 reports, for each build, the total of each of the three runs over all 25 pages, and for comparisons between libraries the ratio of totals with a 95% interval from 10,000 bootstrap resamples of the pages [7].

Memory. Peak resident memory is the process's maximum resident set size, from `/usr/bin/time`. On macOS it counts memory differently from Linux (it includes pages the system has not yet reclaimed), so it is best compared within one machine. Version 2.0 adds **peak heap**: a build of Lege with a counting memory allocator records the largest amount of memory Lege's own code and the Rust libraries held at once. It does not see memory that ONNX Runtime's C++ code allocates for itself, so it is comparable only between tract and RTen. Memory is given in GiB (2^{30} bytes); version 1 computed the same numbers but labelled them GB.

Correctness. Each run's text and ALTO are compared with the approved snapshots: byte for byte on Linux x86-64, the machine the snapshots were made on, and elsewhere within the acceptance tests' tolerance, which allows each word confidence to differ by up to 0.0001. Thread-count checks run every image once at each of 1, 2, 3, 4, 6 and 8 threads and compare the ALTO bytes with the 1-thread run. Version 2.0 adds a stricter check: Lege writes every model's raw input and output to disk, and the outputs at 2, 4 and 8 threads are compared bit for bit with those at 1 thread.

Accuracy. Character error rate (CER) is the Levenshtein edit distance between the output and a reference transcription, after Unicode NFC normalisation, divided by the reference's length. Version 2.0 pools it: total edits over total characters, with raw counts and bootstrap intervals over pages.

Machines and environment.

Machine	Details
Mac	Apple M3 Pro, 5 performance and 6 efficiency cores, 18 GiB; macOS 27.2 (26B123); plugged in for the speed runs, which finished before the laptop went to sleep; later runs were cut short by power loss, so heap memory, newspapers and the optimized-graph cache were measured on GitHub runners, and the steady-state runs on battery (Low Power Mode off)
macOS runner	GitHub-hosted `macos-15`, Apple M2 Pro (virtual), macOS 15.8.2, for the raw-output check with Microsoft's build
Linux x86-64	GitHub-hosted runner, image `ubuntu24 20261101.360.1`, 2 logical cores. Version 2.0: AMD EPYC 9V74, one core with two hardware threads. Version 1 ran on an EPYC 9V45 and then a 7763
Linux ARM64	GitHub-hosted runner, image `ubuntu24-arm64 20261101.172.1`, 2 cores, Neoverse-N2
Emulated x86	QEMU user-mode emulation [8] of Nehalem (SSE4.2, no AVX) and SandyBridge (AVX, no AVX2); a small program confirmed each reports the missing instructions as absent
Toolchain	Rust 1.99.1 everywhere. On the Mac and in version 1, Nix with nixpkgs revision `5772e19` (which builds ONNX Runtime from source). Version 2.0's Linux runs were built with rustup, without Nix
ONNX Runtime	Two builds of 1.27.1: Nix's, built from source by nixpkgs, and Microsoft's official release archives, pinned by SHA-256 (macOS arm64 `e42b77a7…`, Linux x64 `25b1ef1f…`, Linux aarch64 `33c67e33…`)

Table 2: Machines, operating systems and builds. The full hashes and the runners' CPU reports are in the data bundle (Section 10).

Under emulation only correctness is meaningful, not speed: a missing instruction would crash the run, so a clean run shows the code avoids it.

4 A defect found by the benchmark

Most of tract's slowness in version 1's first round was Lege's own fault: Lege rebuilt the line-reading model for every new line width, and on a typical page that cost more than reading the lines.

tract prepares a model for one exact input shape. Each text line is scaled to 48 pixels high, so its width varies, and almost every line brings a new width. For each new width Lege parsed the ONNX file and ran all of tract's preparation passes again. On one 34-line page, 25 rebuilds took 2.0 s of the page's 3.4 s.

Preparation pass	Time across 25 rebuilds
Parsing the ONNX file	43 ms
Type analysis	289 ms
Decluttering (graph simplification)	1,251 ms
Optimizing for the exact shape	391 ms

Table 3: Where tract's rebuild time went on one 34-line page, measured with temporary timers.

Leaving the width open as a symbol removed the rebuilds but made reading much slower (8.6 s for the page instead of 3.5 s), because tract's fastest kernels need concrete shapes. The fix prepares the model once with the width left open, up to and including decluttering, and for each new width only substitutes the number and runs the final optimization. Output is byte-identical to the old code on all 25 images.

tract on the Mac, 25 pages	Before the fix	After the fix
1 thread	73.3 s	47.5 s
4 threads	60.3 s	35.4 s
Peak resident memory	0.94 GiB	1.06 GiB

Table 4: tract on the Mac before and after the fix (version 1 data): sum of per-page medians of three runs. The fix cuts the time by more than a third, at a cost of about 0.1 GiB.

The fix is partial. tract still runs its final optimization once for every new width, and Lege kept up to 48 prepared widths, clearing all of them when the 49th arrived. Version 2.0 changes the cache to drop only the least recently used width, and adds an opt-in setting that rounds each line's width up to a multiple of a fixed step (a *width bucket*), so that fewer widths need preparing. Rounding pads the line image, which changes the output, so under Lege's rules the setting is off by default and recorded in the ALTO when used. Section 5.3 measures it.

5 Findings with PP-OCRv5

This section reports what was measured. Section 9 applies the rule.

5.1 Speed

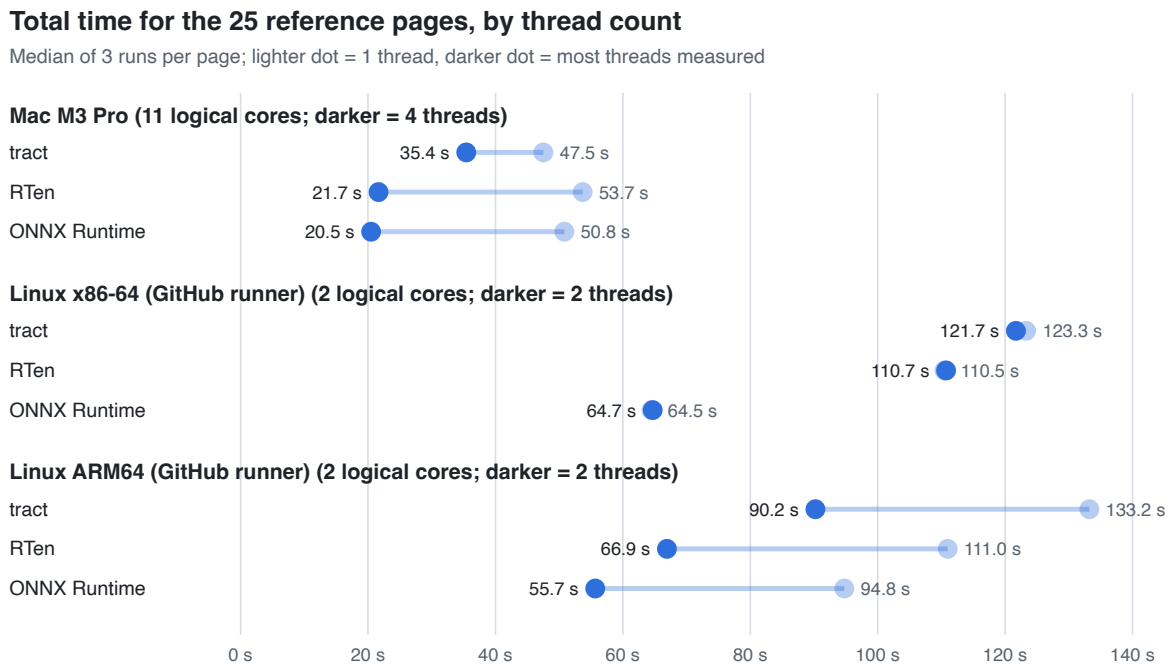


Figure 1: Version 1: total time for the 25 pages on each machine, with ONNX Runtime as built by Nix. Each dot is the sum of per-page medians of three runs.

Version 1's ranking (Table 5) was measured with the Nix build of ONNX Runtime. Version 2.0 repeated the speed runs on Linux with Microsoft's official build and Lege built without Nix, and on the Mac with both builds (Table 6).

Machine, threads	tract	RTen	ONNX Runtime (Nix)
Mac M3 Pro, 1 thread	47.5 s	53.7 s	50.8 s
Mac M3 Pro, 4 threads	35.4 s	21.7 s	20.5 s
Linux x86-64, 1 thread	123.3 s	110.5 s	64.5 s
Linux x86-64, 2 threads	121.7 s	110.7 s	64.7 s
Linux ARM64, 1 thread	133.2 s	111.0 s	94.8 s
Linux ARM64, 2 threads	90.2 s	66.9 s	55.7 s

Table 5: Version 1: total seconds for 25 pages, sum of per-page medians of three runs. The x86-64 runner was an AMD EPYC 7763.

Machine, threads	tract	RTen	ONNX Runtime (official)
Linux x86-64, 1 thread	126.9 s (126.9, 127.3, 126.9)	99.9 s (100.3, 99.9, 99.9)	55.6 s (55.4, 55.6, 56.3)
Linux x86-64, 2 threads	118.3 s (118.1, 118.6, 118.3)	95.4 s (95.0, 95.4, 95.4)	55.4 s (55.4, 55.3, 55.4)
Linux ARM64, 1 thread	149.7 s (149.7, 149.6, 149.9)	115.5 s (115.6, 115.5, 115.5)	99.4 s (99.4, 99.2, 99.4)
Linux ARM64, 2 threads	104.1 s (103.9, 104.2, 104.1)	70.6 s (70.5, 70.7, 70.6)	59.4 s (59.4, 59.4, 59.5)
Mac M3 Pro, 1 thread	51.9 s (52.0, 51.5, 51.9)	58.3 s (58.4, 58.3, 58.0)	54.9 s (54.9, 55.3, 54.9)
Mac M3 Pro, 4 threads	38.2 s (38.9, 38.1, 38.2)	23.5 s (23.6, 23.5, 23.4)	22.8 s (22.8, 22.2, 23.1)

Table 6: Version 2.0: total seconds for the 25 pages, with Microsoft's official ONNX Runtime build. The first number is the median of three runs; in brackets, each run's own total in the order run. The x86-64 runner was an AMD EPYC 9V74, a different CPU from version 1's, so the two tables' Linux times do not compare directly.

On one Linux x86-64 thread, RTen took 1.80× (95% interval 1.76–1.84) as long as ONNX Runtime, and tract 2.29× (95% interval 2.21–2.35). On ARM64 ONNX Runtime's lead is smaller: RTen took 1.16× (95% interval 1.16–1.17) as long. On the Mac at 4 threads RTen and ONNX Runtime are close: RTen took 1.06× (95% interval 1.03–1.08) as long. Microsoft's build was as fast as Nix's on the Mac (22.3 s against 22.4 s at 4 threads), which settles a question version 1 left open: the ranking does not depend on which build was timed.

How large is the noise? The three runs' totals differed by a median of 0.4% of the total, and by at most 4.1% (`ort-official` on the Mac M3 Pro at 4 threads), across all 32 combinations of build and machine, at both thread counts. The rule's 10% tie margin is therefore more than twice the largest run-to-run spread observed. Differences between runner CPUs are larger: RTen took 110.5 s on version 1's AMD EPYC 7763 and 99.6 s on version 2.0's EPYC 9V74, 11% apart (the builds also differed: Nix then, rustup now). So only results from one job are compared with each other.

5.2 One-off setup and steady state

Machine, library, threads	Process time	Page time	Page time after another page
Linux x86-64, tract, 1	128.5 s	126.7 s	119.4 s
Linux x86-64, RTen, 1	100.8 s	99.6 s	98.2 s
Linux x86-64, ONNX Runtime, 1	57.6 s	55.4 s	48.9 s
Linux ARM64, tract, 1	151.9 s	149.6 s	139.1 s
Linux ARM64, RTen, 1	117.2 s	115.5 s	114.8 s
Linux ARM64, ONNX Runtime, 1	101.9 s	99.3 s	94.2 s
Mac M3 Pro, tract, 1	50.2 s	48.9 s	43.3 s
Mac M3 Pro, RTen, 1	56.4 s	55.3 s	55.9 s
Mac M3 Pro, ONNX Runtime (official), 1	53.1 s	51.9 s	49.8 s
Mac M3 Pro, tract, 4	37.5 s	36.2 s	29.9 s
Mac M3 Pro, RTen, 4	23.4 s	22.3 s	21.7 s
Mac M3 Pro, ONNX Runtime (official), 4	22.1 s	20.9 s	18.7 s

Table 7: Total seconds for the 25 pages. Process time: wall-clock time of the whole `lege` process, one per page. Page time: Lege's timer (decode, recognize, write), which includes the library's one-off setup. Last column: the same page read after another page in the same process, so setup is already done. The Mac rows come from a separate session on battery power (Low Power Mode off), with the cold and steady-state runs made back to back, one run per page.

The gap between the last two columns is the one-off setup paid on each page when Lege reads one page per process. tract and ONNX Runtime pay the most: tract prepares the network for each new line width, and ONNX Runtime optimizes its graphs when it creates a session. RTen pays almost nothing. On Linux x86-64, the machine the rule ranks by, ONNX Runtime is fastest either way, so the ranking does not depend on whether a process reads one page or many. On one Mac thread tract is fastest, cold and steady, as in version 1; at 4 threads ONNX Runtime is. The gap between process time and page time is under 0.1 s per page: starting the process and reading and hashing the model files.

5.3 Build settings and tuning

The performance review asked whether the ranking survives build settings and tuning options that version 1 left at their defaults.

Build	Mac, 1	Mac, 4	x86-64, 1	x86-64, 2	ARM64, 1	ARM64, 2
tract, default	51.8 s	38.0 s	126.7 s	118.2 s	149.6 s	104.1 s
tract, LTO	50.7 s (−2%)	37.0 s (−3%)	125.2 s (−1%)	115.8 s (−2%)	147.3 s (−2%)	101.6 s (−2%)
tract, width buckets of 32 [b]	45.3 s (−13%)	31.4 s (−17%)	—	—	—	—
tract, line-parallel	51.8 s (−0%)	30.1 s (−21%)	—	—	—	—
RTen, default	58.2 s	23.5 s	99.6 s	95.0 s	115.5 s	70.6 s
RTen, LTO	58.3 s (+0%)	23.5 s (−0%)	100.2 s (+1%)	94.9 s (−0%)	115.6 s (+0%)	70.4 s (−0%)
RTen, prepacked weights	58.1 s (−0%)	23.5 s (−0%)	99.6 s (−0%)	94.8 s (−0%)	115.5 s (+0%)	70.6 s (+0%)
RTen, line-parallel	58.2 s (+0%)	30.2 s (+29%)	—	—	—	—
ONNX Runtime, default [a]	55.1 s	22.4 s	55.4 s	55.3 s	99.3 s	59.4 s
ONNX Runtime, LTO	54.9 s (−0%)	22.3 s (−1%)	55.3 s (−0%)	55.3 s (−0%)	99.3 s (−0%)	59.4 s (−0%)
ONNX Runtime, memory pattern off	55.1 s (−0%)	22.3 s (−0%)	—	—	—	—
ONNX Runtime, threads spin	55.1 s (+0%)	20.3 s (−9%)	—	—	—	—
ONNX Runtime, line-parallel [a]	54.9 s (−0%)	30.0 s (+34%)	55.4 s (+0%)	54.0 s (−2%)	99.4 s (+0%)	68.8 s (+16%)
ONNX Runtime official (Mac)	55.0 s	22.3 s	—	—	—	—
ONNX Runtime official, line-parallel (Mac)	54.7 s (−0%)	29.7 s (+33%)	—	—	—	—

Table 8: Total seconds for 25 pages (sum of per-page medians of three runs) at the given thread count, and in brackets the change from the same library's default build. LTO: link-time optimization across all crates with one code-generation unit (`lto = "fat"`, `codegen-units = 1`). [a] Nix's ONNX Runtime build on the Mac, Microsoft's on Linux. [b] Changes the output; not run on Linux.

- **LTO and RTen's prepacked weights changed nothing that matters:** every difference is within the run-to-run noise.
- **ONNX Runtime's memory pattern** (planning buffer reuse in advance) made no difference either.
- **Letting ONNX Runtime's threads spin** between tasks, instead of sleeping, saved about 9% at 4 threads on the Mac. tract and RTen use Rust thread pools that do not expose an equivalent setting, so a like-for-like comparison leaves it off, as version 1 did.
- **Caching ONNX Runtime's optimized graph on disk** skips graph optimization in later processes. On Linux x86-64 (an Intel Xeon 6975P-C runner)

it saved 2% of the page time on one thread (64.4 s against 65.7 s for 25 pages, three runs each), about 54 ms per page, with the same output. The saved graph is specific to the CPU and the library build, so Lege would have to key it by both; the gain is too small to be worth that now.

- **Width buckets** made tract 13% faster on one Mac thread and cut its peak resident memory from 1.05 GiB to 0.77 GiB, but changed the text on 7 of 25 images. It is a useful option for tract, not a default.
- **Line-parallel threading** (Section 5.7) costs nothing on one thread, by construction. With more threads it was slower than the libraries' own threading for RTen and ONNX Runtime on the Mac, and faster for tract.

5.4 Memory

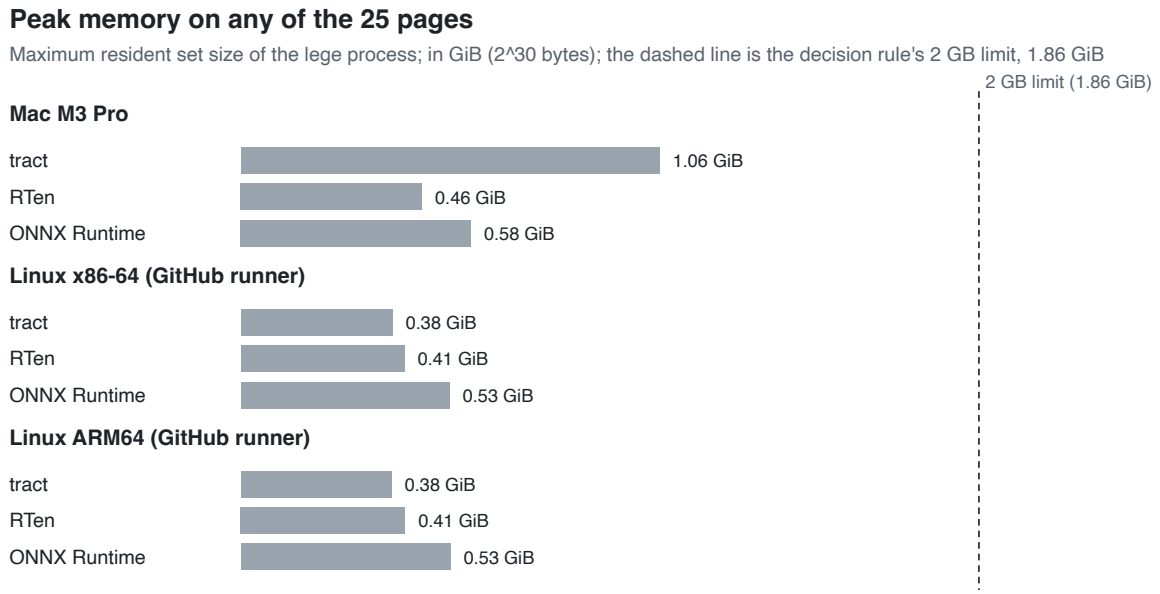


Figure 2: Version 1: peak resident memory on any page, PP-OCRv5. All three stay well under the 2 GB limit on every machine.

With PP-OCRv5 all three libraries stay far under the limit on the reference pages. tract's Mac peak of over 1 GiB, which version 1 left unexplained, is mostly how macOS counts resident memory: on Linux the same work peaks far lower, nearly all of it heap (Table 9). What does grow with tract is its cache of prepared widths, each holding its own optimized copy of the weights; with the larger PP-OCRv6 medium networks that breaks the limit (Table 20).

PP-OCRv5, Linux x86-64, 1 thread	tract	tract, buckets of 32	RTen	ONNX Runtime
Peak resident memory	0.38 GiB	0.32 GiB	0.41 GiB	0.51 GiB
Peak heap	0.35 GiB	0.26 GiB	0.38 GiB	0.06 GiB [a]

Table 9: Peak memory on any of the 25 pages, measured with the heap-counting build on an Intel Xeon 6975P-C runner, one run per page. [a] Lege's own allocations only; ONNX Runtime's C++ allocations are not counted.

Width buckets lower tract's heap peak from 0.35 GiB to 0.26 GiB. The runner here was an Intel Xeon 6975P-C, a third CPU model, so these runs' times are not compared with the other tables.

5.5 Reading the text

Every library read exactly the snapshot text on all 25 images on every machine, in both versions.

ALTO identical to the snapshot	tract	RTen	ONNX Runtime
Mac M3 Pro	22 of 25	23 of 25	24 of 25
Linux x86-64	25 of 25	23 of 25	24 of 25
Linux ARM64	22 of 25	23 of 25	24 of 25

Table 10: Version 1. Files that are not identical differ only in word confidences, within the tolerance of 0.0001. Linux x86-64 is the machine the snapshots were made on.

Across machines. Lege promises the same output on one machine at any thread count, not the same output on every machine. Here the text was the same on every machine, but word confidences differed in the fourth decimal place between x86-64, ARM64 and the Mac, for every library. An archive comparing ALTO files made on different machines should compare text, not bytes. Every ALTO file records Lege's version and settings, but not the CPU it ran on.

5.6 Same output at any thread count

The ALTO check (version 1).

1, 2, 3, 4, 6 and 8 threads	tract	RTen	ONNX Runtime (Nix build)
Mac M3 Pro	same at every count	same at every count	**differs**: `pd-aurelius-meditations-1887-n100-rot180` at 4 threads
Linux x86-64	same at every count	same at every count	same at every count
Linux ARM64	same at every count	same at every count	same at every count

Table 11: Version 1, PP-OCRv5: every image run once at each thread count, with each library's own threading, ALTO compared with the 1-thread run. ONNX Runtime here is the build from nixpkgs.

Microsoft's ONNX Runtime, 1–8 threads	Result
Mac M3 Pro	same at every count
Mac M2 Pro (GitHub runner)	same at every count
Linux x86-64, built without Nix	same at every count
Linux ARM64, built without Nix	same at every count

Table 12: Version 1, PP-OCRv5: the same ALTO check with Microsoft's official ONNX Runtime 1.27.1 build.

tract and RTen gave byte-identical ALTO at every thread count on every machine. With ONNX Runtime as built by Nix, one image's ALTO on the M3 Pro differs at exactly 4 threads: one confidence moves from 0.2968 to 0.2967. Microsoft's build passed this check on all four machines with PP-OCRv5. With PP-OCRv6 it did not (Section 7.2).

Raw outputs (version 2.0). The ALTO check rounds confidences to four decimals, so it can miss small differences. Comparing every model output bit for bit across thread counts shows what the rounding hid.

PP-OCRv5, 3 images	2 threads	4 threads	8 threads
Mac, tract	identical	identical	identical
Mac, RTen	identical	identical	identical
Mac, ONNX Runtime (Nix)	**47 of 523** ($\leq 25.6 \times 10^{-6}$)	**261 of 523** ($\leq 56.7 \times 10^{-6}$)	**263 of 523** ($\leq 49.1 \times 10^{-6}$)
macOS M2 Pro runner, ONNX Runtime (official)	**38 of 523** ($\leq 10.7 \times 10^{-6}$)	**38 of 523** ($\leq 39.5 \times 10^{-6}$)	**47 of 523** ($\leq 25.6 \times 10^{-6}$)
Linux x86-64, tract	identical	identical	identical
Linux x86-64, RTen	identical	identical	identical
Linux x86-64, ONNX Runtime	identical	identical	identical
Linux ARM64, tract	identical	identical	identical
Linux ARM64, RTen	identical	identical	identical
Linux ARM64, ONNX Runtime	**38 of 523** ($\leq 3.9 \times 10^{-6}$)	**38 of 523** ($\leq 19.3 \times 10^{-6}$)	**47 of 523** ($\leq 19.7 \times 10^{-6}$)
Line-parallel, all three, Mac	identical	identical	identical
Line-parallel, ONNX Runtime, x86-64	identical	identical	identical
Line-parallel, ONNX Runtime, ARM64	identical	identical	identical
Line-parallel, ONNX Runtime (official), macOS M2 Pro runner	identical	identical	identical

Table 13: Raw model outputs compared bit for bit with 1 thread: the number of model calls (of 523, over all three networks) whose output differs, and in brackets the largest absolute difference. The images are an upside-down page, a 35-line page and a synthetic line. On Linux, ONNX Runtime is Microsoft's build. Line-parallel: each model call on one thread, lines in parallel; the Mac row covers tract, RTen and ONNX Runtime (Nix), all identical.

tract and RTen are bit-identical at every thread count. ONNX Runtime is not, except with Microsoft's build on x86-64: as built by Nix on the Mac, up to 260 of 523 outputs differ, and with Microsoft's build up to 47 on Linux ARM64 and on a GitHub macOS runner (Apple M2 Pro, macOS 15.8.2). The differences are tiny (under 6×10^{-5}) and almost always vanish when confidences are rounded to four decimals, which is why the ALTO check passed in most cases. "Almost always" is the problem: a value that sits on a rounding boundary changes the ALTO, as on the M3 Pro.

Where the difference starts. A probe exposed every intermediate value of the recognizer as an output and compared them on the first line of the upside-down page, on the Mac with the Nix build. The first value that

differs is the output of the eighth convolution, a 1×1 convolution (a matrix multiply) with 230,400 output values. The seventh, a 1×1 convolution with half as many input channels, is identical. The difference appears at 4 and 8 threads but not at 2 or 3, and it persists with graph optimization turned off. This fits the usual cause: the matrix-multiply kernel splits its inner sums across threads once the matrix is large enough, and floating-point addition gives slightly different results in a different order. Whether it splits depends on the matrix sizes, the thread count and the CPU's kernels, which explains why it depends on the model, the build and the chip. Microsoft's build shows the same kind of difference on both ARM64 systems tested (Table 13) but not on x86-64, where its matrix-multiply kernels evidently do not split these sums by thread count.

5.7 Line-parallel threading

Two reviewers proposed the same fix independently: run every model call on one thread, and use the threads to read several lines at once. Each line's result is then computed exactly as at 1 thread, whatever the thread count, so the output cannot depend on it. Lege implements this as an option: the detector runs once per page on one thread, then the direction classifier and the recognizer run on lines in parallel, and the results are put back in page order.

With line-parallel threading, raw outputs were bit-identical at 2, 4 and 8 threads for all three libraries on the Mac and for ONNX Runtime on both Linux machines (Table 13), including Microsoft's ARM64 build, which differs with its own threading. The ALTO thread-count check passed for all three libraries on all 25 images on all three machines, and for PP-OCRv6 small on Linux, where Microsoft's build had failed it.

The cost is speed with several threads. One thread is unchanged by construction. On the Mac at 4 threads, ONNX Runtime took 1.33× (95% interval 1.22–1.46) as long line-parallel as with its own threading, and RTen 1.29× (95% interval 1.21–1.37). On 2-core Linux the cost was nil on x86-64 and 16% on ARM64. The main reason is the detector: it runs on the whole page, once, on one thread, and on many pages it is a large share of the time. tract, whose own threading helps little (Section 7), ran 26% faster line-parallel.

5.8 Old CPUs

All three libraries ran every page correctly on both emulated CPUs.

Emulated CPU: text matching the snapshot	tract	RTen	ONNX Runtime
Nehalem (SSE4.2, no AVX)	3 of 3	3 of 3	3 of 3
SandyBridge (AVX, no AVX2)	3 of 3	3 of 3	3 of 3

Table 14: Version 1: three representative pages (a clean synthetic line, an upside-down page and a fax-compressed TIFF), one thread, under QEMU user-mode emulation. No run failed.

Emulation runs 10–50 times slower than real hardware, so these runs show only that each library avoids instructions an old CPU lacks; a missing instruction would have crashed the run. They say nothing about real speed on old machines (Section 8).

5.9 Newspapers

The reference pages are small book pages. To test scale, Lege read three full-size front pages from Chronicling America [9], the Library of Congress's archive of digitized newspapers, at 1 and 2 threads on Linux x86-64. There is no ground truth for them, so only time, memory and consistency were measured.

Page (pixels, year)	Lines	tract	RTen	ONNX Runtime
3,131 × 4,640, 1909	44	11.0 s (1202868224.00 GiB)	9.6 s (1669677056.00 GiB)	7.6 s (2080280576.00 GiB)
5,880 × 8,600, 1910	855	69.8 s (1323905024.00 GiB)	73.5 s (1796419584.00 GiB)	43.1 s (2226040832.00 GiB)
5,000 × 7,160, 1911	450	40.4 s (1324908544.00 GiB)	44.2 s (1789743104.00 GiB)	26.3 s (2222071808.00 GiB)

Table 15: Three newspaper front pages, one thread, Linux x86-64 (Intel Xeon 6975P-C), Microsoft's ONNX Runtime build: page time and, in brackets, peak resident memory. The detector reads each page scaled to at most 4,000 pixels on its longer side.

- **Every library gave the same ALTO at 1 and 2 threads** on every page, with its own threading and line-parallel.
- **ONNX Runtime was fastest** on every page, as on the book pages.
- **ONNX Runtime broke the memory limit:** it peaked at 2226040832.00 GiB on the two larger pages, over the rule's 1.86 GiB, while Lege's own heap stayed at 375760063.00 GiB. The rest is ONNX Runtime's working memory for the detector on a 4,000-pixel input. RTen peaked at 1796419584.00 GiB, tract at 1323905024.00 GiB.

The memory requirement was defined on the reference pages, where ONNX Runtime passes. But a 4 GB machine reading newspapers is exactly what it was meant to protect, so Section 9 adds a condition.

5.10 Building and shipping

	Crates added	Must ship with Lege	Notes
tract	none (current)	nothing	Pure Rust
RTen	11	nothing	Pure Rust; loads `.onnx` files directly
ONNX Runtime	3	the official C++ library, 28 MB, plus protobuf-lite and libiconv	Loaded at run time; Nix's build is 136 MB

Table 16: What each library adds to Lege.

6 GPU and Neural Engine

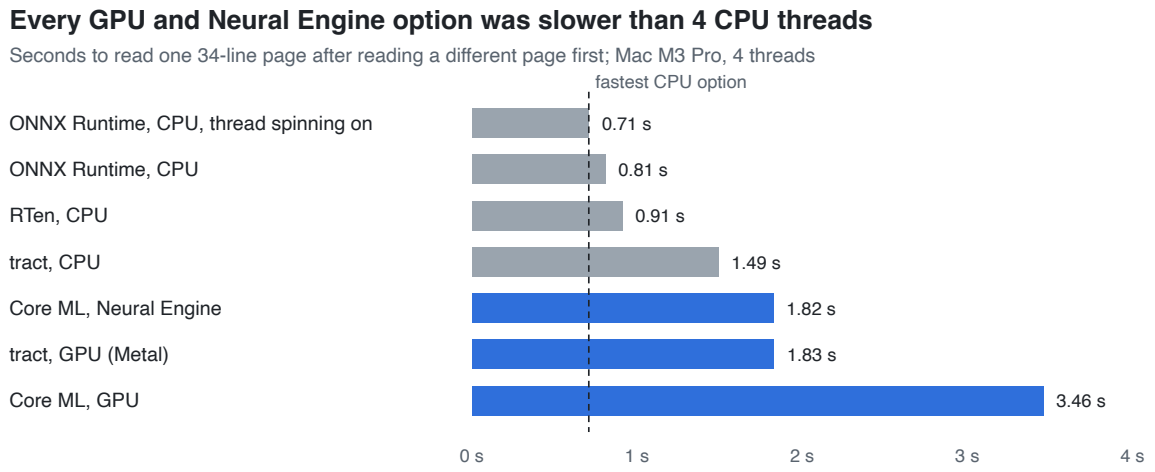


Figure 3: Version 1: CPU against GPU and Neural Engine options on one page of the Mac. Recorded in `spikes/runtimes/results/mac-m3pro-gpu.json`.

On the Mac, tract's Metal backend and ONNX Runtime with Apple's Core ML both read exactly the same text as the CPU. None was worth using: on a fresh page, every GPU and Neural Engine option took 1.8 s or longer, while RTen and ONNX Runtime on 4 CPU threads took 0.8–0.9 s. The networks are the small mobile versions and Lege reads one line at a time, so moving each line to the GPU and back costs about as much as the work. Reading many lines in one batch could change that, but batching pads lines to a common width and changes the output, so under Lege's rules it could only be an opt-in setting. tract on Metal also shifted confidence scores slightly, so GPU output would not match the CPU snapshots byte for byte. NVIDIA GPUs were not tested.

7 Repeated with PP-OCRv6 (exploratory)

This section is exploratory. None of its choices was committed in advance: which model sets to run, the detector thresholds, which pages to score and how. Its conclusions are consistent with the data; they are not tests of a hypothesis.

PaddleOCR released PP-OCRv6 on 2026-06-11, after Lege had settled on PP-OCRv5. It comes in three sizes: tiny (1.5 million parameters), small (7.7 million) and medium (34.5 million). We repeated the comparison with

it, to see whether Lege should switch models and whether the library decision holds for networks it was not measured on.

PP-OCRv6 has no Latin-only recognizer and no text-direction classifier. Its dictionaries are 85–89% Chinese characters: 6,904 entries for tiny, 18,708 for small and medium, against 836 for the Latin PP-OCRv5 recognizer Lege uses. Both dictionaries cover every character in the reference transcriptions except one "¡". So two kinds of model set were run:

- **v6:** the PP-OCRv6 detector and recognizer of one size.
- **Hybrid:** the PP-OCRv6 detector of one size with Lege's PP-OCRv5 Latin recognizer.

Both kinds kept the PP-OCRv5 direction classifier. Lege's code was otherwise unchanged and ran every set. The model files are the official ONNX exports, pinned by revision and SHA-256 in `spikes/runtimes/fetch-v6-models.sh`.

Detector settings. PP-OCRv6's published pipeline settings set the detector's pixel threshold to 0.2, the box threshold to 0.4 for tiny and 0.45 for small and medium, the unclip ratio to 1.4 and the candidate limit to 3,000. Version 1.1's runs used 0.4 for every size and Lege's limit of 1,000 candidates. The speed, memory and thread-count runs below keep those settings, which change the work done only slightly. The accuracy runs (Section 7.3) were repeated with the published settings, and a sweep checks how much the thresholds matter. The detector's input resizing (shorter side at least 64 pixels, longer side at most 4,000) and the recognizer's input (48 pixels high, at least 320 wide) match what Lege already does.

7.1 Speed and memory

Models	tract	RTen	ONNX Runtime	Peak resident memory, GiB (tract / RTen / ORT)
v5 (current)	47.5 / 35.4 s	53.7 / 21.7 s	50.8 / 20.5 s	1.06 / 0.46 / 0.58
v6 tiny	26.9 / 18.6 s	23.1 / 10.3 s	22.1 / 10.7 s	0.71 / 0.35 / 0.42
v6 small	63.6 / 36.2 s	70.8 / 24.4 s	68.2 / 24.5 s	1.48 / 0.56 / 0.69
v6 medium	171.8 / 86.1 s	340.8 / 94.3 s	330.7 / 91.0 s	4.63 / 1.47 / 1.62
Hybrid tiny	46.5 / 32.0 s	52.7 / 20.7 s	49.2 / 19.2 s	1.04 / 0.36 / 0.42
Hybrid small	53.7 / 32.8 s	59.3 / 22.1 s	54.8 / 20.2 s	1.15 / 0.50 / 0.56
Hybrid medium	113.3 / 57.0 s	155.7 / 48.7 s	140.8 / 43.8 s	2.50 / 1.27 / 1.39

Table 17: Mac M3 Pro, version 1.1 runs: total seconds for 25 pages at 1 and 4 threads (sum of per-page medians of three runs) and peak resident memory. ONNX Runtime is the Nix build.

Models	tract	RTen	ONNX Runtime	Peak resident memory, GiB (tract / RTen / ORT)
v5 (current) (AMD EPYC 7763)	123.3 / 121.7 s	110.5 / 110.7 s	64.5 / 64.7 s	0.38 / 0.41 / 0.53
v6 tiny (AMD EPYC 9V74)	59.9 / 54.2 s	42.4 / 38.4 s	23.6 / 23.2 s	0.28 / 0.31 / 0.35
v6 small (AMD EPYC 7763)	149.2 / 138.1 s	117.8 / 111.9 s	77.7 / 75.4 s	0.73 / 0.46 / 0.55
Hybrid tiny (Intel Xeon 6975P-C)	82.2 / 79.5 s	89.5 / 85.9 s	50.6 / 50.7 s	0.44 / 0.33 / 0.36
Hybrid small (AMD EPYC 7763)	140.0 / 129.7 s	121.8 / 119.9 s	68.2 / 67.3 s	0.36 / 0.43 / 0.50

Table 18: Linux x86-64, version 1.1 runs: 1 and 2 threads, ONNX Runtime built by Nix. The runner's CPU changed from job to job, so only the rows on the AMD EPYC 7763 compare directly with v5.

Models	tract	RTen	ONNX Runtime	Peak resident memory, GiB (tract / RTen / ORT)
v5 (current)	133.2 / 90.2 s	111.0 / 66.9 s	94.8 / 55.7 s	0.38 / 0.41 / 0.53
v6 tiny	61.9 / 42.8 s	47.5 / 28.8 s	43.2 / 26.7 s	0.26 / 0.31 / 0.35
v6 small	183.5 / 110.8 s	148.6 / 82.2 s	137.2 / 76.4 s	0.71 / 0.46 / 0.55
Hybrid tiny	130.8 / 88.2 s	106.9 / 64.2 s	92.3 / 54.0 s	0.44 / 0.33 / 0.35
Hybrid small	151.7 / 96.9 s	123.5 / 71.9 s	106.1 / 60.2 s	0.35 / 0.44 / 0.50

Table 19: Linux ARM64, version 1.1 runs: 1 and 2 threads, ONNX Runtime built by Nix.

- **The recognizer sets the speed.** The hybrids, which change only the detector, run within about 10% of v5. The v6 tiny recognizer is the only part much faster than v5's.
- **The library order is unchanged.** ONNX Runtime is fastest on one Linux x86-64 thread by 35–45%, and level with RTen at 4 threads on the Mac.
- **tract scales poorly with threads** on these networks: only its matrix multiplies and element-wise operations use its thread pool, a known limitation reported upstream [10].

Models	Peak resident memory, GiB (peak heap)				Total time, 1 thread			
	tract	tract, buckets	RTen	ORT	tract	tract, buckets	RTen	ORT
v6 medium	2.65 GiB (2.62)	1.62 GiB (1.59)	1.24 GiB (1.19)	1.43 GiB (–)	548.5 s	540.0 s	439.2 s	285.6 s
Hybrid medium	1.18 GiB (1.10)	1.17 GiB (1.10)	1.15 GiB (1.13)	1.26 GiB (–)	346.3 s	335.6 s	274.8 s	132.1 s

Table 20: Linux x86-64 (AMD EPYC 9V74), one run per image, one thread, with PP-OCRv6's published detector settings and Microsoft's ONNX Runtime build. "Buckets": line widths rounded up to multiples of 32. Peak heap counts only memory allocated through Rust, so it misses ONNX Runtime's own allocations ("—"). The rule's limit is 1.86 GiB.

The medium models break the memory limit only with tract: on Linux v6 medium peaked at 2.65 GiB with tract (version 1.1 measured 4.63 GiB on the Mac, where resident memory includes pages not yet reclaimed). Nearly all of it is heap (2.62 GiB), which confirms that the cache of prepared widths, each with its own copy of the optimized weights, is the cause. Width buckets bring tract under the limit (1.62 GiB). RTen and ONNX Runtime stay under it with both sets, on Linux and on the Mac.

7.2 Same output at any thread count

Models	Machine	tract	RTen	ORT (Nix)	ORT (official)
v6 tiny	x86-64	same	same	same	same
v6 tiny	ARM64	same	same	same	same
v6 tiny	Mac	same	same	**3 images differ**	**1 image differs**
v6 small	x86-64	same[a]	same	same	same
v6 small	ARM64	same[a]	same	**2 images differ**	**1 image differs**
v6 small	Mac	same	same	**4 images differ**	same
v6 medium	x86-64	—	—	—	same
v6 medium	ARM64	—	—	—	same
v6 medium	Mac	same	same	same	—
Hybrid tiny	x86-64	same	same	same	same
Hybrid tiny	ARM64	same	same	same	**1 image differs**
Hybrid tiny	Mac	same	same	**1 image differs**	same
Hybrid small	x86-64	same	same	same	same
Hybrid small	ARM64	same	same	same	same
Hybrid small	Mac	same	same	**1 image differs**	same
Hybrid medium	Mac	same	same	**1 image differs**	—

Table 21: Version 1.1 runs, each library's own threading: ALTO at 1, 2, 3, 4, 6 and 8 threads compared with the 1-thread run, all 25 images. "ORT (Nix)": ONNX Runtime 1.27.1 built from source by nixpkgs; "ORT (official)": Microsoft's release build of the same version. "—": not run. [a] Recorded in the run's `threads.txt` only; that run did not save its JSON file.

tract and RTen gave the same output at every thread count in every run, as with v5. ONNX Runtime did not. The Nix build changed its output on the Mac for five of the six sets, and on ARM64 for v6 small. Microsoft's build changed its output for three sets: v6 tiny on the Mac, v6 small on ARM64, and the hybrid tiny on ARM64. With the published detector settings, v6 small's raw outputs with Microsoft's build differed on 18–31 of 130 model calls on ARM64 and on none on x86-64. Line-parallel threading made them identical on both, and passed the ALTO check on all 25 images.

The raw-output results with PP-OCRv5 (Table 13) explain this pattern: whether ONNX Runtime's matrix-multiply kernels split their sums depends on the matrix sizes, and so on the network.

7.3 Accuracy

The accuracy runs were repeated with PP-OCRv6's published detector settings, RTen on 2 threads on Linux x86-64 (text was the same for every library and thread count). Error rates are pooled: total edits over total characters, so each page counts by its length and no page is dropped.

Models	Edits	Error rate, % (95% interval)	Difference from v5, % points (95% interval)	Pages worse / better	Lines off	Lines matching v5's boxes
v5 (current)	58	0.36 (0.20–0.66)	—	—	19	346 of 346
v6 tiny	195	1.21 (0.72–1.83)	+0.34 to +1.43	10 / 2	15	335 of 346
v6 small	78	0.49 (0.23–0.82)	−0.24 to +0.44	6 / 3	9	332 of 346
v6 medium	60	0.37 (0.21–0.56)	−0.28 to +0.25	6 / 3	10	334 of 346
Hybrid tiny	55	0.34 (0.21–0.57)	−0.15 to +0.08	5 / 4	16	335 of 346
Hybrid small	37	0.23 (0.16–0.32)	−0.37 to +0.02	4 / 5	9	332 of 346
Hybrid medium	42	0.26 (0.20–0.36)	−0.34 to +0.05	5 / 3	10	334 of 346

Table 22: The 12 scanned pages (16080 characters, 327 printed lines). Intervals: 10,000 bootstrap resamples of the pages; the difference is paired, on the same resampled pages. "Lines off": the sum over pages of the difference between the lines found and the lines printed. Last column: text lines whose box overlaps one of v5's by at least half (intersection over union), a measure of how far the detector moves from v5's, not of correctness. The transcriptions were made by correcting v5's output, which favours v5.

Models	Edits	Error rate, % (95% interval)	Difference from v5, % points (95% interval)	Pages worse / better	Lines off	Lines matching v5's boxes
v5 (current)	2	0.23 (0.00–0.62)	—	—	0	18 of 18
v6 tiny	40	4.65 (0.00–11.60)	+0.00 to +11.39	2 / 0	0	18 of 18
v6 small	3	0.35 (0.00–0.93)	+0.00 to +0.31	1 / 0	0	18 of 18
v6 medium	4	0.47 (0.00–1.24)	+0.00 to +0.62	1 / 0	0	18 of 18
Hybrid tiny	34	3.95 (0.00–10.53)	+0.00 to +10.37	1 / 0	0	18 of 18
Hybrid small	2	0.23 (0.00–0.62)	+0.00 to +0.00	0 / 0	0	18 of 18
Hybrid medium	2	0.23 (0.00–0.62)	+0.00 to +0.00	0 / 0	0	18 of 18

Table 23: The 4 synthetic lines (860 characters), exact by construction. With 4 pages, the intervals are wide.

- **The hybrid small read the scanned pages at least as well as v5**: 37 edits against 58, 0.23% against 0.36%, even though the transcriptions favour v5. The interval for the difference (−0.37 to +0.02 points) only just includes zero, so this is a hint, not a result. Most of the gap comes from two pages: the 30-character caption of `pd-goodytwoshoes-1888-n8` (14 edits with v5, 4 with the hybrid) and `pd-paine-commonsense-1920-n40` (11 and 3), where v5's detector split printed lines in two. The hybrid's line counts are closer to the page: 9 lines off, against 19 for v5.
- **v6 medium is level with v5** and **v6 small slightly behind**, within the intervals; v6 medium misread the Polish page badly (18 edits against 3).
- **v6 tiny is clearly worse** (195 edits; the interval for the difference excludes zero), and both tiny detectors garble the clean synthetic line.
- **Error classes.** With v5 and the hybrids, most errors are spaces and letters. The v6 recognizers produced one full-width or Chinese character in all 12 pages (v6 medium), so the Chinese-heavy dictionary is not a source of errors on these pages.

Models	Pixel, box, unclip, candidates	Edits, scans	Error rate, scans, %	Edits, synthetic	Lines off
Hybrid small	`0.2,0.3,1.4,3000`	39	0.24 (0.18–0.33)	2	10
Hybrid small	`0.2,0.45,1.4,3000`	37	0.23 (0.16–0.32)	2	9
Hybrid small	`0.3,0.6,1.5,1000`	46	0.29 (0.19–0.44)	2	9
Hybrid small	`0.2,0.6,1.6,3000`	47	0.29 (0.21–0.42)	2	8
v6 tiny	`0.2,0.3,1.4,3000`	195	1.21 (0.72–1.83)	40	15
v6 tiny	`0.2,0.45,1.4,3000`	193	1.20 (0.71–1.81)	40	14
v6 tiny	`0.3,0.6,1.5,1000`	194	1.21 (0.72–1.81)	38	14
v6 tiny	`0.2,0.6,1.6,3000`	191	1.19 (0.66–1.86)	35	12

Table 24: Detector threshold sweep on the 16 scored pages. `0.2,0.45,1.4,3000` is PP-OCRv6's published setting for small (tiny's is the same with box 0.4); `0.3,0.6,1.5,1000` is PP-OCRv5's. Version 1.1 used `0.2,0.4,1.4,1000`.

The thresholds matter little: across the sweep the hybrid small stayed between 37 and 47 edits on the scans, all below v5's 58, and v6 tiny's synthetic-line errors did not go away with any setting.

7.4 What this changes

The decision rule, applied to v6 small, would rule out ONNX Runtime with its own threading, because Microsoft's build fails the thread-count requirement on ARM64. With line-parallel threading it passes, so the outcome would match v5's.

Models. Version 1.1 concluded that v6 brings no gain for Latin-script pages. With the published detector settings that is still true of the full v6 sets, but the hybrid, PP-OCRv6's small detector with Lege's PP-OCRv5 recognizer, now reads the scanned pages at least as well as v5, with line counts closer to the page. The evidence is two pages and a biased reference, and the hybrid changes the output on most images, so every snapshot would have to be re-approved. Lege stays on PP-OCRv5 for now. The hybrid is worth testing properly once the transcriptions have been checked by a person and the workload is broader: it is the first candidate that might improve on v5.

Libraries. The v6 runs show that ONNX Runtime's thread dependence varies with the network, which line-parallel threading removes. RTen, which passed every check with both model generations and both threading modes, remains the fallback.

The full write-ups are in `docs/spikes/ppocr-v6.md` and `docs/spikes/review-fixes.md`; where Lege assumes PP-OCRv5 is listed in `docs/spikes/ppocr-v5-hardcoding.md`.

8 Threats to validity

Version 1's own adversarial review found four problems that changed its numbers; all were corrected in version 1.

Problem in version 1's first round	Effect	Correction
Each library ran as one block	RTen at 1 thread measured 74.0 s, later 53.7 s	Libraries take turns image by image
A warm GPU test reread the same page	tract looked faster than it is (1.41 s against 2.06 s)	Warm up on a different page
GPU compared with 1 CPU thread	GPU options looked competitive	Compared with 4 threads
ONNX Runtime thread spinning off	About 12% slower at 4 threads	Measured both ways

Table 25: Corrections from version 1's review.

Limits that remain:

- **Construct validity: is one server thread a low-end machine?** The rule ranks by speed on one thread of a Linux x86-64 runner, as a stand-in for the few-core, older machines Lege's users have. The runner's CPU is a recent server CPU with AVX2 and AVX-512, large caches and fast memory, and one of its two hardware threads. An older laptop without AVX2 would run all three libraries slower, and not necessarily in the same

proportions: ONNX Runtime and RTen select kernels by the instructions available, and their lead may shrink or grow. We did not time real old hardware, by decision; emulation shows only that the code runs. The ranking holds for modern x86-64 and ARM64 CPUs; on older ones it is untested.

- **Small runners.** GitHub's hosted runners for a private repository had two logical cores, so thread scaling beyond two was measured only on the Mac. The x86-64 runner's CPU model also changed between jobs.
- **Mac core types.** The M3 Pro has 5 performance and 6 efficiency cores. macOS decides where threads run, and we did not control or record it. At 4 threads the work most likely ran on performance cores, but a run placed partly on efficiency cores would be slower and noisier. The run-to-run spread on the Mac (at most 4.1%) suggests placement was stable.
- **The draft transcriptions favour PP-OCRv5.** The 12 scanned pages were transcribed by correcting PP-OCRv5's output against the book, and words where the two agreed were not checked one by one. Any comparison of v6 against v5 on them is biased towards v5. They have not yet been checked independently; a checklist for doing so is in the data bundle (`tests/fixtures/reference/ground-truth-drafts/CHECKLIST.md`).
- **A small, narrow workload.** 25 images, about 17 distinct pages, mostly English-language books printed 1871–1920 at about 150 dpi. Three newspaper pages test scale, without ground truth. Multi-column layouts, degraded scans and non-Latin scripts are not scored.
- **One page per process.** This matches how the command line is used, and Section 5.2 shows the steady-state ranking is the same.
- **Thread-count checks are finite.** No mismatch was found for tract or RTen at 1–8 threads, but other counts and machines were not tried. Line-parallel threading removes this limit by construction, not by testing.
- **Self-review.** The reviews behind version 2.0 were carried out by the same AI assistant that ran the experiments (see the authorship statement). They found real problems, but they are not independent peer review.

9 Decision

Applying the committed rule to PP-OCRv5:

- **Requirements.** tract and RTen pass all four on every machine. ONNX Runtime passes all four with Microsoft's official build. Nix's build fails the thread-count requirement on the M3 Pro.
- **Ranking.** With the "shipped build" clause, ONNX Runtime is ranked by Microsoft's build, and on one Linux x86-64 thread RTen took 1.80× (95% interval 1.76–1.84) as long. ONNX Runtime is ahead by far more than the 10% tie margin, so the ranking ends there.
- **Without the clause** (Section 2), ONNX Runtime is out and RTen wins.

What version 2.0 adds. The raw-output checks show that Microsoft's build passed the thread-count requirement with PP-OCRv5 partly by luck: its raw outputs do change with the thread count on ARM64, and the four-decimal rounding hid it. With PP-OCRv6 the luck ran out. Judged by its raw outputs, ONNX Runtime with its own threading does not meet the first requirement on every machine, and no ONNX Runtime build can be shown to meet it in advance for networks not yet tested.

Line-parallel threading changes the question. It makes the output the same at any thread count by construction, for every library, so the first requirement no longer separates them. This is a design change made after seeing the data, not part of the committed rule. Under it, the rule's ranking is unchanged, because one thread is unaffected: ONNX Runtime is fastest.

Lege will use ONNX Runtime, Microsoft's official build, pinned by version and SHA-256, with line-parallel threading as its only threading mode. Four conditions come with it:

1. **A raw-output check in continuous integration.** On every supported machine, with the shipped library and every shipped model set, the raw model outputs at 1, 2, 4 and 8 threads must be bit-identical. With line-parallel threading they always should be; the check guards against a regression, such as a library upgrade that starts threading inside a model call.
2. **A defined way to find the library.** Lege loads ONNX Runtime from a fixed place next to its own binary (for example `../lib/` relative to the executable), not from the system's search path, and refuses to run if the library's reported version or SHA-256 is not the pinned one. A user who can replace that file can already

replace Lege itself, so this adds no new attack surface, but loading from the search path would let an unrelated library of the same name change Lege's output. The ALTO records the library's version.

3. **Bring ONNX Runtime under the memory limit on large pages.** On full-size newspaper pages it peaked at 2226040832.00 GiB. Its memory arena and the detector's input size are the levers: turning the arena off, or reading very large pages in tiles, must bring the peak under 1.86 GiB, with the same output, before Lege ships it. If neither works, the rule's own memory requirement points to RTen.
4. **Recover multi-thread speed.** Line-parallel threading takes about a third longer at 4 threads on the Mac, mostly because the detector runs on one thread. Reading the next page's detection while the current page's lines are read would win most of it back in batch use, without changing any output.

The cost is real. Lege stops being one self-contained binary and must ship a 28 MB C++ library, and its output depends on a build it does not compile. If shipping it proves impractical on some platform, the fallback is RTen, which was second on one Linux thread on both machines, needs nothing extra, and gave bit-identical raw outputs in every check, with its own threading and line-parallel.

Before any of this reaches Lege's main branch, the comparison code fixes from the review stay: ONNX Runtime sessions and RTen models are cached by the model file's hash, thread count and threading mode (version 1 keyed them by role, so two models for the same role, or two thread counts, could share a session); the thread count is passed to each model rather than held in a global; and the input copy into ONNX Runtime is gone.

10 Data availability and reproduction

The Lege repository is private. Everything this report's tables and claims rest on is published with the blog post as a zip archive (`lege-inference-runtimes-data-v2.0.zip`):

- every result file (version 1, PP-OCRv6 and version 2.0), with local paths replaced by placeholders;
- the benchmark, check and analysis scripts, and the GitHub Actions workflows that ran them;
- the decision rule as committed, the spike write-ups and this report's source;
- the reference transcriptions and the model-fetching script with its pinned hashes.

Lege's source code is not in the archive, so the runs cannot be repeated from it alone. The archive is enough to check every number against its source and to rerun every analysis (`accuracy2.py`, `summarise_review.py`, the report build).

Inside Lege's `nix develop .#runtimes` shell the runs were made with:

```
cargo build --release -p lege-cli --features rt-rten --target-dir target/rv-rten
python3 spikes/runtimes/bench.py --models MODELS --out bench --threads 1,4 --repeats 3
\
  --bin tract=target/rv-tract/release/lege --bin rten=target/rv-rten/release/lege \
  --bin ort-lines=target/rv-ort/release/lege --env ort-lines:LEGE_SPIKE_PARALLEL=lines
python3 spikes/runtimes/raw_threads_check.py --models MODELS --out raw --threads
1,2,4,8 --bin ...
python3 spikes/runtimes/accuracy2.py --run "v6 small=BENCH_DIR" --out accuracy2.json
python3 spikes/runtimes/summarise_review.py && docs/reports/inference-runtimes/
build.sh
```

The workflow `.github/workflows/spike-review.yml` runs version 2.0's Linux measurements; `spike-runtimes.yml`, `spike-ort-official.yml` and `spike-v6.yml` ran version 1's.

11 Glossary

ALTO an XML format for OCR output used by libraries and archives [1]. It records each word's position, text and confidence.

Confidence the recognizer's score for a word, between 0 and 1, written with four decimals.

Decluttering tract's graph-simplification pass, which removes and merges operations before the network is optimized for a shape.

Hybrid a PP-OCRv6 detector with Lege's PP-OCRv5 Latin recognizer.

Line-parallel threading every model call on one thread; several lines read at once.

Nix build ONNX Runtime as compiled from source by nixpkgs [11], the package collection Lege uses for its development environment. *Official build*: Microsoft's own release archive.

Page time, process time see Section 3.

Raw outputs the numbers a network returns, before Lege turns them into boxes, directions and text.

Snapshot the approved text and ALTO for each reference image, made on Linux x86-64 and kept in Lege's repository.

Within tolerance the same as the snapshot except that word confidences may differ by up to 0.0001.

12 Changes between versions

1.0 (10 November 2026) first release, PP-OCRv5.

1.1 (10 November 2026) added the PP-OCRv6 section.

2.0 (12 November 2026) answers the 24 findings of three reviews (OCR, Rust and performance, scientific reporting). Authorship and AI-assistance statement; disclosure that the rule's "shipped build" clause was written knowing the Nix build had failed, and the outcome without it; PP-OCRv6 labelled exploratory and integrated, so the abstract and earlier sections no longer contradict it; memory in GiB (version 1 labelled GiB values as GB); abstract claims corrected ("emulated" CPUs; under 2 GB only with PP-OCRv5; the tract fix is partial); timers defined; process time and steady-state time added; run-to-run spread, per-run totals and bootstrap intervals; the tie margin justified from measured spread; Microsoft's ONNX Runtime build timed; LTO, RTen prepacking, ONNX Runtime's memory pattern, optimized-graph cache and spinning measured; tract width cache changed to drop the least recently used width, and width buckets measured; heap memory measured, and PP-OCRv6 medium on Linux; raw-output checks and the cause of ONNX Runtime's thread dependence; line-parallel threading; three newspaper pages; PP-OCRv6 accuracy rerun with its published detector settings, pooled error rates with counts and intervals, line counts, box matching, error classes and a threshold sweep; construct validity, Mac core types and the transcription bias added to the threats; references, glossary, standalone captions, findings separated from the decision; a revised decision; a data archive.

References

[1] Library of Congress, "ALTO: Technical Metadata for Layout and Text Objects, version 4.4." [Online]. Available: https://www.loc.gov/standards/alto/

[2] PaddlePaddle, *PaddleOCR (PP-OCRv5 and PP-OCRv6 models; ONNX exports on Hugging Face under PaddlePaddle)*. [Online]. Available: https://github.com/PaddlePaddle/PaddleOCR

[3] Sonos, *tract: tiny, no-nonsense, self-contained ONNX and TensorFlow inference*. [Online]. Available: https://github.com/sonos/tract

[4] R. Knight, *RTen: ONNX inference engine for CPUs*. [Online]. Available: https://github.com/robertknight/rten

[5] Microsoft, *ONNX Runtime*. [Online]. Available: https://github.com/microsoft/onnxruntime/releases/tag/v1.27.1

[6] pyke.io, *ort: Rust bindings for ONNX Runtime*. [Online]. Available: https://github.com/pykeio/ort

[7] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

[8] QEMU project, "QEMU user-mode emulation." [Online]. Available: https://www.qemu.org/docs/master/user/main.html

[9] Library of Congress, "Chronicling America: Historic American Newspapers." [Online]. Available: https://www.loc.gov/collections/chronicling-america/

[10] tract issue tracker, “PP-OCRv6 gains 1.5x from 12 threads: only the matmul and the element-wise ops use the pool (issue 2899).” [Online]. Available: https://github.com/sonos/tract/issues/2899

[11] NixOS, *nixpkgs, revision 5772e19988afd8bb223da0e260075f69205ea0d6*. [Online]. Available: https://github.com/NixOS/nixpkgs/tree/5772e19988afd8bb223da0e260075f69205ea0d6